

Discussion 4

—
Jett Rosen - Sections 13 and 14



All feelings are temporary, you are loved,
you are enough, and you matter.

Logistics

- Attendance QR code will be on the last slide, make sure to fill it out!
- HW4 is out
 - Start early!

Exit Ticket Reflection

- Thanks for the great recommendations, I'm happy to learn my discussions have good taste!
 - A lot of great things that I have seen, like Attack on Titan, The Prestige, Fullmetal Alchemist Brotherhood (the third time), Project Hail Mary, Bleach, Fight Club, Everything Everywhere All at Once
 - Some ones that are definitely on my watchlist: Mulholland Drive, Rosemead, Limitless
 - Shoutout whoever said "Hm all bad tbh"
 - And more!
- More importantly, thank you for the feedback on what was useful and what I can do better!

More DP!

- Useful tip: The number of dimensions in your memo = the number of variables in your recurrence.
 - Think about it: we need to track every combination of input variables to store what their output was so that we can use it later
- I cannot stress enough: **take a modular approach** (follow the recipe)

The Recipe

1. Define **value version** of what we want to compute
2. Derive a **recurrence** for this value version
3. **Size of table:** How many dimensions? Range of each dimension?
4. What are the **base case(s)**?
5. To fill in a cell, which other cells need to be filled already? **In which order do I fill the table?**
6. Which cell(s) contain the **final answer**?
7. **Running time** = (size of table) • (time to fill each entry)
8. To **reconstruct the solution** (instead of just its value) follow arrows from final answer to base case

Graph DP

- Two main problems that we focus on:
 - Single-Source Shortest Paths (SSSP): Given a vertex v , find a shortest path from v to all other vertices
 - Algorithm: Bellman-Ford
 - All-Pairs Shortest Paths (APSP): Find the shortest path between *every* pair of vertices
 - Algorithm: Floyd-Warshall
- Key Observations:
 - If the shortest s - t path passes through v , then P must be the shortest s - v path followed by the shortest v - t path.
 - The shortest path will not contain a cycle (there are no negative-weight cycles in the graph)

Bellman-Ford for SSSP

- **Definition:** Let $\text{dist}^{(i)}(s, v)$ be the length of a shortest s -to- v path that uses **at most** i edges, or ∞ if there's no such path.
 - In the course notes, the recurrence is different. Correct, but different.
- **Key insight:** If a shortest path from s to t goes through vertex v , then it must be a shortest path from s to v , then a shortest path from v to t .
 - Also, every path from s to t must include a path from s to some neighbor of t , u .
- **Recurrence:**

$$\text{dist}^{(i)}(s, v) = \min \left\{ \text{dist}^{(i-1)}(s, v), \min_{(u,v) \in E} \{ \text{dist}^{(i-1)}(s, u) + w(u, v) \} \right\}$$

$$\text{dist}^{(0)}(s, s) = 0$$

$$\text{For all } v \neq s, \text{dist}^{(0)}(s, v) = \infty$$

Floyd-Warshall for APSP

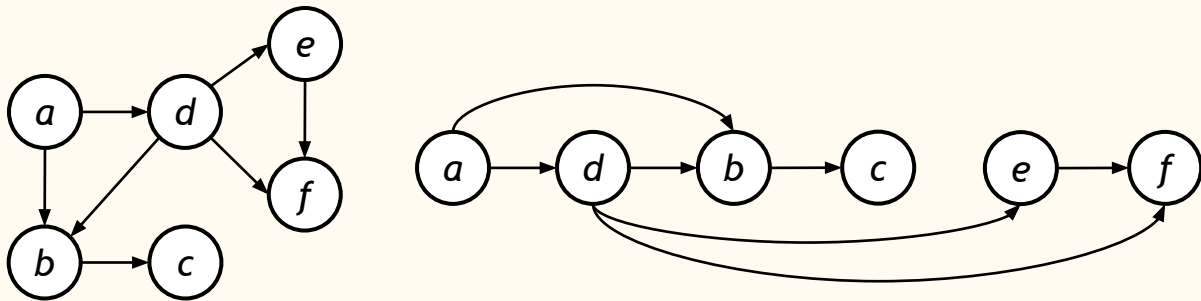
- **Definition:** Let $\text{dist}^{(i)}(s, v)$ **be the** length of shortest s - t path that is **allowed** to use vertices $s, t \cup v_1, \dots, v_i$ (not necessarily all of them).
- **Key insight:** If a shortest path from s to t goes through vertex v , then it must be a shortest path from s to v , then a shortest path from v to t .
- **Recurrence:**

$$\text{dist}^{[i]}(s, t) = \min \left\{ \text{dist}^{[i-1]}(s, i) + \text{dist}^{[i-1]}(i, t), \quad \text{dist}^{[i-1]}(s, t) \right\}$$

$$\text{dist}^{[0]}(s, t) = \begin{cases} 0 & \text{if } s = t \\ w(s, t) & \text{if } (s, t) \in E \\ \infty & \text{otherwise} \end{cases}$$

Topological Ordering

- **Definition:** *Topological ordering* for a Directed Acyclic Graph (DAG) is a **linear ordering** of vertices such that for every directed edge (u, v) , vertex u comes **before** v in the ordering



10

- This allows us to assign rows/columns of a table to the vertices in a DAG

Common Subproblems Dependencies

(that you have seen so far)

1

"Shrink and recurse"

Involves "shrinking" the problem's range and recursing into solving the smaller range.

Typically, the range is defined by indices, and the subproblems depend on smaller sections of the range (e.g., prefix and/or suffix).

Examples: Sequence Problems: LCS, LAS

2

"Use it or lose it"

Involves decisions of choosing or skipping items and recursing into different cases.

Typically, the subproblems depend on the remaining items and/or the modified constraints (e.g., reduced capacity, remaining sum, or elements to process).

Examples: 0-1 Knapsack, All Pairs Shortest Path (Floyd-Warshall)

WARNING: This is NOT an exhaustive list of the type of problems you will see in EECS

Common Subproblems Dependencies

(that you have seen so far)

3

"How did I get here?"

Involves **checking states that could have lead to the current state**

Typically, given a current state, the number of ways to get there is the number of ways we could have gotten to a predecessor state

Examples: Climb Stairs, Tiling Problem, Knights Walk

WARNING: This is NOT an exhaustive list of the type of problems you will see in EECS

Greedy Algorithms

- Requires that **locally optimal decisions** lead to **globally optimal results**
 - To prove this, we use an exchange argument
- Exchange Argument Ingredients:
 - $ALG = \{s_1, s_2, \dots, s_m\}$ is the series of decisions made by the algorithm
 - OPT is an **arbitrary** optimal solution $\rightarrow$ we know **nothing** about how it was created, only that it is **an** optimal solution
 - We consider the case where $ALG \neq OPT$: Let s_{diff} denote the first different selection between ALG and OPT
 - We want to exchange an element of OPT with s_{diff} and prove it is still **some** optimal solution, OPT' (formally, repeating this step is induction)
 - Finally, we show that ALG cannot be improved without violating constraints of problem

Exchange Step

- ▶ Notation:

$ALG = \{s_1, \dots, s_m\}$ = algorithm's (ordered) output; OPT = some *arbitrary* optimal solution

- ▶ If $ALG = OPT$, done. Otherwise,

- ▶ Let s_{diff} be the first element in ALG that **differs** from OPT (this means $s_1, \dots, s_{\text{diff}-1}$ agree with OPT)
- ▶ Modify OPT to agree with ALG up to s_{diff} (this involves modifying some $f \in OPT$ that is **not** in $s_1, \dots, s_{\text{diff}-1}$)
- ▶ Let's call this modified solution OPT'

- ▶ Now, we need to prove that

- ▶ OPT' is still a **valid** solution
- ▶ OPT' is still an **optimal** solution

Typically involve some
problem-specific/
algorithm-specific
observations



Exit Ticket

